Informe final

NetDiscreta
Simulador de envío de mensajes en una red aplicando Matemáticas Discretas

Neyder Javier Rojas Lasso
Ingeniería de Sistemas
Universidad Simón Bolívar
Asignatura: Matemáticas Discretas
Docente: _____

Noviembre de 2026

Resumen

Se desarrolló NetDiscreta, una aplicación de escritorio en Java (Swing) que simula el envío de un mensaje binario por una red de computadores. El proyecto integra los temas del curso en un solo flujo: la red se modela como un grafo ponderado y la ruta se calcula con Dijkstra; los equipos se agrupan y operan como conjuntos; el producto cartesiano entre grupos define una relación de enlaces directos; el mensaje es una cadena sobre el alfabeto {0, 1} que se valida contra lenguajes formales; los errores del canal se modelan con un espacio muestral y una variable aleatoria binomial; y la detección de errores se realiza con un bit de paridad calculado mediante álgebra booleana (XOR). El sistema tiene 31 clases y unas 3.500 líneas de código, organizadas en capas de modelo, lógica y vista. Las pruebas confirmaron que los resultados del programa coinciden con los cálculos hechos a mano y que la simulación converge a los valores teóricos.

Palabras clave: grafos, conjuntos, producto cartesiano, lenguajes formales, distribución binomial, álgebra de Boole, bit de paridad, Java Swing.

1. Introducción

Las matemáticas discretas son la base de la computación: las redes son grafos, los datos son cadenas de símbolos, los errores de transmisión obedecen leyes de probabilidad y los circuitos digitales operan con álgebra de Boole. Sin embargo, en el curso estos temas suelen estudiarse por separado, con ejercicios en papel, y es difícil ver cómo se conectan en un problema real.

Este proyecto propone un escenario común para todos los temas: enviar un mensaje de un computador a otro y comprobar si llegó bien. Cada etapa del envío corresponde a un tema y se implementa como un módulo interactivo con su propia visualización.

2. Objetivos

2.1 Objetivo general

Desarrollar una aplicación de escritorio en Java con NetBeans y JFrame que simule la transmisión de mensajes binarios en una red de computadores, integrando conjuntos, producto cartesiano, alfabetos, cadenas y lenguajes, probabilidad, distribución binomial, grafos y álgebra booleana.

2.2 Objetivos específicos y cumplimiento

#	Objetivo específico	Estado	Dónde
1	Modelar la red como grafo ponderado y calcular la ruta más corta con Dijkstra.	Cumplido	Módulo 1
2	Implementar operaciones entre conjuntos de nodos y mostrarlas en diagramas de Venn.	Cumplido	Módulo 2
3	Generar el producto cartesiano entre grupos y representarlo en el plano cartesiano.	Cumplido	Módulo 3
4	Representar mensajes como cadenas sobre Σ = {0, 1} y validar su pertenencia a un lenguaje.	Cumplido	Módulo 4
5	Modelar errores con espacio muestral, eventos y distribución binomial; comparar teoría y simulación.	Cumplido	Módulo 5
6	Detectar errores con bit de paridad (XOR) y mostrar su tabla de verdad.	Cumplido	Módulo 6

3. Marco teórico

3.1 Grafos y ruta más corta

Un grafo G = (V, E) está formado por un conjunto de vértices V y un conjunto de aristas E. En un grafo ponderado cada arista tiene un peso; aquí el peso es la latencia del enlace. El **grado** de un vértice es el número de aristas que inciden en él, y se cumple el teorema del apretón de manos:

$$\Sigma_{v \in V} \, gr(v) = 2|E|$$

Un grafo es **conexo** si existe un camino entre cualquier par de vértices; el programa lo comprueba con un recorrido en anchura (BFS). La **matriz de adyacencia** M tiene $M_{ij} = 1$ si hay arista entre i y j. El **algoritmo de Dijkstra** encuentra el camino de menor peso desde un origen: en cada paso elige el vértice no visitado con menor distancia acumulada y actualiza (relaja) la distancia de sus vecinos.

3.2 Conjuntos

Un conjunto es una colección de elementos sin repetición ni orden. Dado el universo U (todos los equipos), se definen:

Operación	Notación	Definición
Unión	A ∪ B	{x | x ∈ A ∨ x ∈ B}
Intersección	A ∩ B	{x | x ∈ A ∧ x ∈ B}
Diferencia	A − B	{x | x ∈ A ∧ x ∉ B}
Diferencia simétrica	A Δ B	(A − B) ∪ (B − A)
Complemento	A'	{x ∈ U | x ∉ A}
Subconjunto	A ⊆ B	todo elemento de A está en B

Para la cardinalidad de la unión se usa el principio de inclusión–exclusión: $|A \cup B| = |A| + |B| - |A \cap B|$.

3.3 Producto cartesiano y relaciones

El producto cartesiano de A y B es el conjunto de pares ordenados $A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$, con $|A \times B| = |A| \cdot |B|$. Como el orden importa, en general $A \times B \neq B \times A$. Una **relación** binaria R de A en B es un subconjunto de $A \times B$. En el proyecto, $(a, b) \in R$ si los equipos a y b tienen un enlace directo.

3.4 Alfabetos, cadenas y lenguajes

Un **alfabeto** Σ es un conjunto finito y no vacío de símbolos. Una **cadena** w es una secuencia finita de símbolos de Σ; la cadena vacía se escribe λ. Σ^* es el conjunto de todas las cadenas sobre Σ y Σ^n el de las cadenas de longitud n, con $|\Sigma^n| = |\Sigma|^n$. Operaciones: longitud $|w|$, concatenación $w \cdot v$, potencia w^n ($w^0 = \lambda$), inversa w^R, prefijos y sufijos. Un **lenguaje** es cualquier subconjunto $L \subseteq \Sigma^*$. El programa usa cuatro lenguajes: longitud fija, paridad par, cabecera y cola, y palíndromos ($w = w^R$).

3.5 Probabilidad y distribución binomial

El experimento es enviar n bits; cada bit llega correcto (C) o con error (E). El **espacio muestral** es $\Omega = \{C, E\}^n$, con $|\Omega| = 2^n$. Un **evento** es un subconjunto de Ω, por ejemplo «a lo sumo un error». La **variable aleatoria** $X: \Omega \to \mathbb{N}$ asigna a cada resultado su número de errores. Si cada bit falla de forma independiente con probabilidad p, X sigue una distribución binomial:

$$P(X = k) = C(n, k) \cdot p^k \cdot (1 - p)^{n-k}, \text{ con } C(n, k) = n! / (k!(n - k)!)$$

$$E[X] = n \cdot p \quad \mathrm{Var}(X) = n \cdot p \cdot (1 - p) \quad \sigma = \sqrt{\mathrm{Var}(X)}$$

Por la ley de los grandes números, al repetir el experimento muchas veces la frecuencia relativa de un evento se acerca a su probabilidad; el módulo Canal lo comprueba con simulación.

3.6 Álgebra de Boole y bit de paridad

El álgebra de Boole trabaja con los valores {0, 1} y las operaciones NOT (A'), AND (A·B), OR (A + B) y XOR ($A \oplus B = A \cdot B' + A' \cdot B$). Dos expresiones son equivalentes si tienen la misma tabla de verdad; así se comprueban leyes como De Morgan, $(A + B)' = A' \cdot B'$. Toda función puede escribirse como **suma de productos canónica**, sumando los minitérminos de las filas donde vale 1.

La **paridad par** usa XOR: el emisor calcula $P = b_1 \oplus b_2 \oplus \ldots \oplus b_n$ y envía $b_1 \ldots b_n P$, que siempre tiene una cantidad par de unos. El receptor calcula el síndrome $S = r_1 \oplus \ldots \oplus r_{n+1}$: si $S = 1$ hubo un número impar de errores. Si el número de errores es par, $S = 0$ y el error pasa sin detectarse; su probabilidad es $P(X = 2) + P(X = 4) + \ldots$

4. Diseño del sistema

4.1 Arquitectura

El código se organiza en capas. El **modelo** guarda los datos, la **lógica** contiene los algoritmos matemáticos sin depender de la interfaz, y la **vista** construye las ventanas y los dibujos. Esta separación permitió probar la lógica de forma independiente antes de conectarla a la interfaz.

Paquete	Responsabilidad	Clases
main	Punto de entrada	NetDiscreta
modelo	Datos del dominio y estado compartido	Nodo, Arista, Grafo, Conjunto, Estado
logica	Algoritmos y cálculos matemáticos	Dijkstra, OperacionesConjuntos, ProductoCartesiano, Cadenas, Lenguaje, Binomial, Evento, EspacioMuestral, SimuladorCanal, Paridad, ExpresionBooleana
vista	Ventana principal y un panel por módulo	VentanaPrincipal, PanelRed, PanelGrupos, PanelConexiones, PanelMensajes, PanelCanal, PanelVerificador, Estilo, Refrescable
vista.graficos	Dibujos con Graphics2D	LienzoGrafo, LienzoVenn, LienzoPlano, LienzoBarras, LienzoCircuito

4.2 Flujo de datos entre módulos

Todos los módulos comparten un objeto `Estado` con la red, los grupos, el mensaje y el último envío. Cuando el usuario entra a una pestaña, la ventana llama a su método `refrescar()` (interfaz `Refrescable`), de modo que cada módulo ve los cambios hechos en los demás.

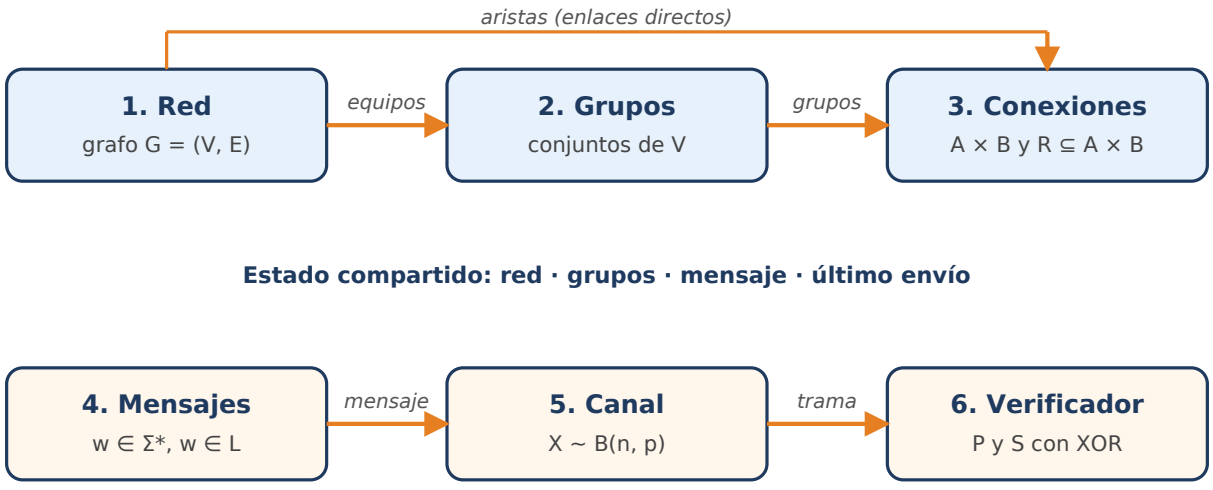


Figura 1. Flujo de información entre los seis módulos.

4.3 Decisiones de diseño

- **Interfaz escrita en código** (sin el diseñador visual de NetBeans) para controlar el diseño y que el proyecto sea fácil de leer y versionar en Git.
- **Dibujo propio con Graphics2D** en lugar de librerías externas: el proyecto no tiene dependencias y se abre directamente en NetBeans.
- **Proyecto Maven**, que NetBeans reconoce con solo el archivo pom.xml.
- **Lenguajes como clases**: cada lenguaje sabe validar una cadena y explicar por qué pertenece o no, lo que facilita agregar lenguajes nuevos.
- **Expresiones booleanas con un analizador de descenso recursivo**, que respeta la precedencia NOT > AND > XOR > OR y acepta AND implícito (AB).

5. Implementación

A continuación se muestran los fragmentos centrales de algunos algoritmos.

5.1 Dijkstra (clase Dijkstra)

```
while (visitados.size() < g.getNodos().size()) {
    // 1. Elegir el nodo no visitado con menor distancia
    Nodo u = null;
    for (Nodo n : g.getNodos())
        if (!visitados.contains(n) && (u == null || dist.get(n) < dist.get(u))) u = n;
    if (u == null || dist.get(u) == Integer.MAX_VALUE) break;
    visitados.add(u);
    // 2. Relajar las aristas que salen de u
    for (Arista ar : g.getAristas()) {
        if (!ar.contiene(u)) continue;
        Nodo v = ar.otro(u);
        int nueva = dist.get(u) + ar.getPeso();
        if (nueva < dist.get(v)) { dist.put(v, nueva); previo.put(v, u); }
    }
}
```

5.2 Probabilidad binomial (clase Binomial)

```
public static double combinaciones(int n, int k) {
    k = Math.min(k, n - k);
    double r = 1;
    for (int i = 1; i <= k; i++) r = r * (n - k + i) / i;   // evita calcular factoriales grandes
    return Math.round(r);
}
public double probabilidad(int k) {           // P(X = k)
    return combinaciones(n, k) * Math.pow(p, k) * Math.pow(1 - p, n - k);
}
```

5.3 Bit de paridad (clase Paridad)

```
public static int xorTotal(String bits) {
    int r = 0;
    for (char c : bits.toCharArray()) r ^= (c == '1' ? 1 : 0);   // r = r XOR bit
    return r;
}
public static String agregarParidad(String m) { return m + xorTotal(m); }   // emisor
public static int sindrome(String trama)    { return xorTotal(trama); }     // receptor
```

5.4 Gramática de las expresiones booleanas (clase ExpresionBooleana)

```
disyuncion := xor ( '+' xor )*
xor        := conjuncion ( '⊕' | '^' conjuncion )*
conjuncion := unario ( ['·' | '*'] unario )*        // AB = A·B
unario     := '!' unario | primario ( ''' )*        // !A  o  A'
primario   := LETRA | '0' | '1' | '(' disyuncion ')'
```

6. Pruebas

Cada resultado del programa se comparó con el valor calculado a mano. Las pruebas usaron la red de ejemplo (6 equipos y 7 enlaces), los grupos Norte = {PC1, PC2, R1, R3} y Sur = {R2, R3, SRV}, y el mensaje w = 1011001.

#	Módulo	Caso de prueba	Esperado (a mano)	Obtenido	✓
1	Red	Ruta más corta PC1 → SRV	PC1–R1–R3–R2–SRV, 4+2+3+4 = 13 ms	13 ms, misma ruta	✓
2	Red	Suma de grados	$2\|E\| = 2 \cdot 7 = 14$	14	✓
3	Red	Ruta hacia un equipo aislado	No existe camino	«No existe camino»	✓
4	Grupos	Norte ∩ Sur	{R3}	{R3}	✓
5	Grupos	|Norte ∪ Sur|	$4 + 3 - 1 = 6$	6	✓
6	Conexiones	|Norte × Sur| y |R|	$4 \cdot 3 = 12$; R = {(R1,R2), (R1,R3), (R3,R2), (R3,SRV)}	12 y |R| = 4	✓
7	Mensajes	1011001 ∈ L (paridad par)	4 unos → pertenece	w ∈ L	✓
8	Mensajes	$\|\Sigma^3\|$ y $\|L \cap \Sigma^3\|$	$2^3 = 8$; {000, 011, 101, 110} → 4	8 y 4	✓
9	Canal	P(X = 0), n = 8, p = 0,05	$0{,}95^8 = 0{,}663420$	0,663420	✓
10	Canal	P(X ≤ 1), n = 8, p = 0,05	0,663420 + 0,279335 = 0,942755	0,942755	✓
11	Canal	Suma de P(X = k), k = 0…8	1	1,000000	✓
12	Canal	Simulación de 100.000 envíos	≈ 66 342 sin errores	66 467 (0,13 % de diferencia)	✓
13	Verificador	Bit de paridad de 1011001	4 unos → P = 0; trama 10110010	P = 0, 10110010	✓
14	Verificador	Trama con 1 error (10100010)	S = 1, detectado	S = 1	✓
15	Verificador	Trama con 2 errores (10100110)	S = 0, no detectado	S = 0, «Error NO detectado»	✓

#	Módulo	Caso de prueba	Esperado (a mano)	Obtenido	✓
16	Verificador	¿(A + B)' ≡ A'·B'? (De Morgan)	Equivalentes	Equivalentes	✓
17	Verificador	¿A + B·C ≡ (A + B)·C?	No equivalentes (A=1, C=0)	No equivalentes	✓
18	Verificador	A ⊕ B ⊕ C en minitérminos	Σm(1, 2, 4, 7)	Σm[1, 2, 4, 7]	✓

También se probaron entradas inválidas: nombres de equipo repetidos, conexiones de un equipo consigo mismo, símbolos fuera del alfabeto, mensajes no binarios y expresiones mal escritas. En todos los casos el programa muestra un aviso claro y no se cierra.

7. Resultados

Las siguientes capturas muestran cada módulo funcionando con los datos de prueba.

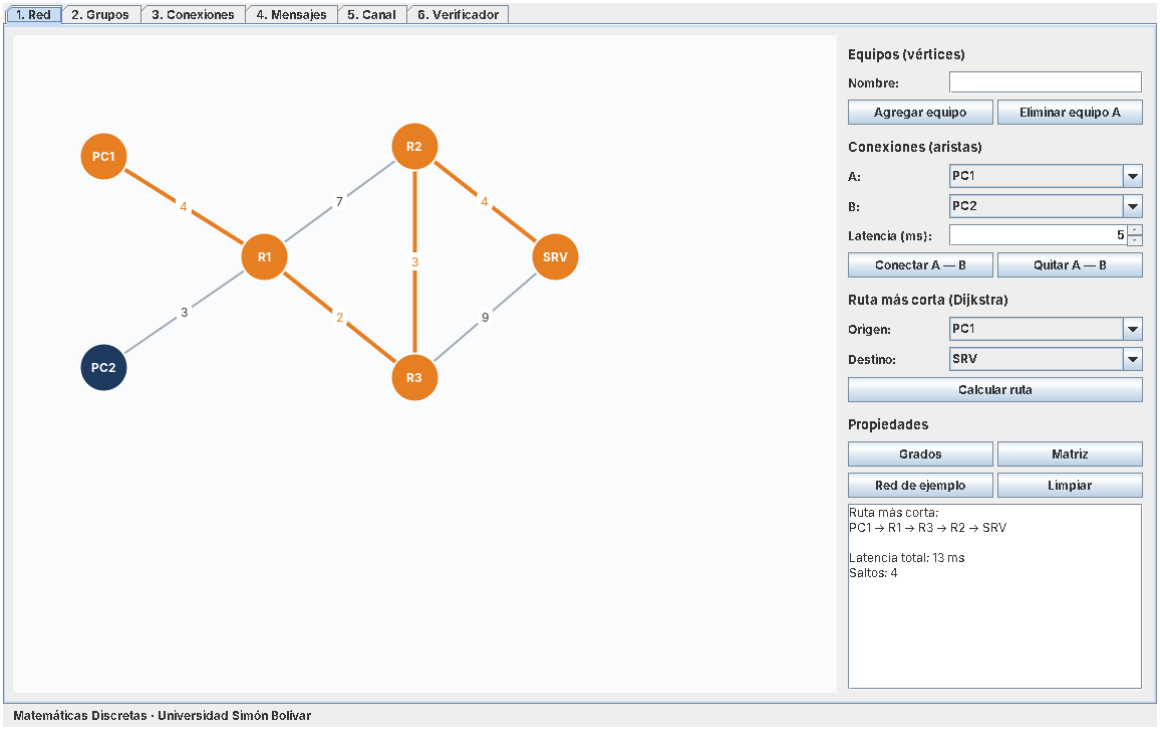


Figura 2. Módulo Red: ruta más corta de PC1 a SRV (13 ms) resaltada en naranja.

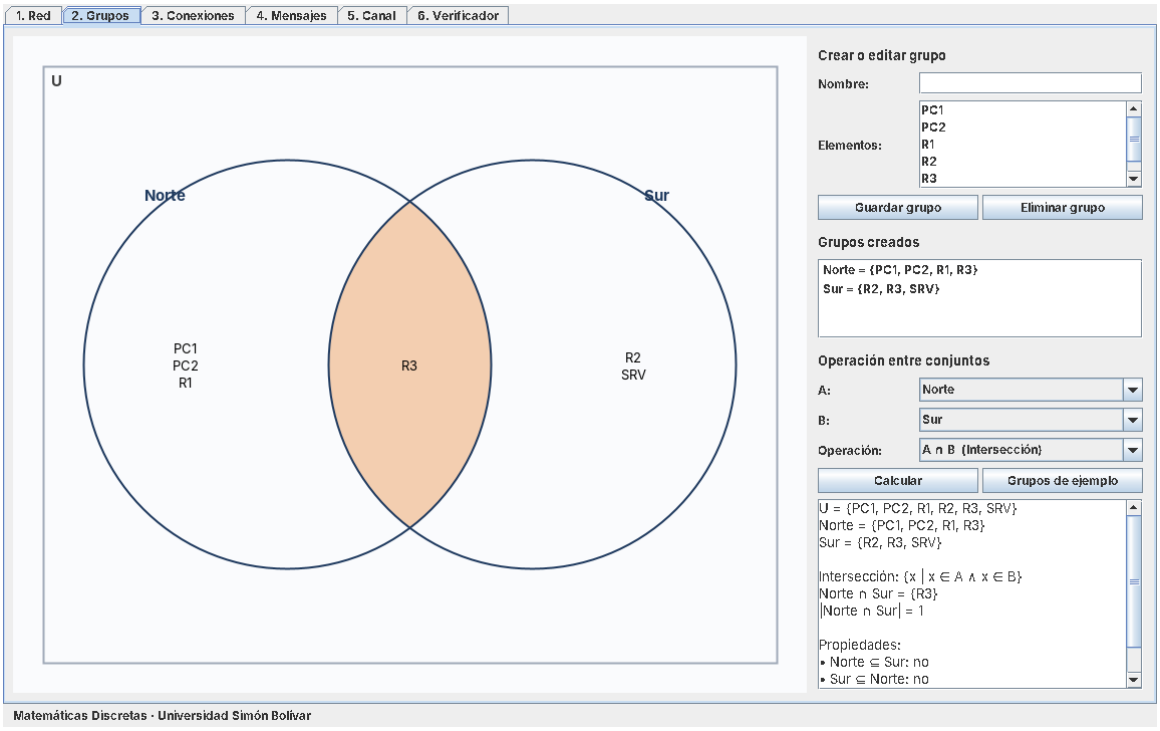


Figura 3. Módulo Grupos: diagrama de Venn de Norte ∩ Sur = {R3}.



Figura 4. Módulo Conexiones: Norte × Sur en el plano; en naranja, la relación R.

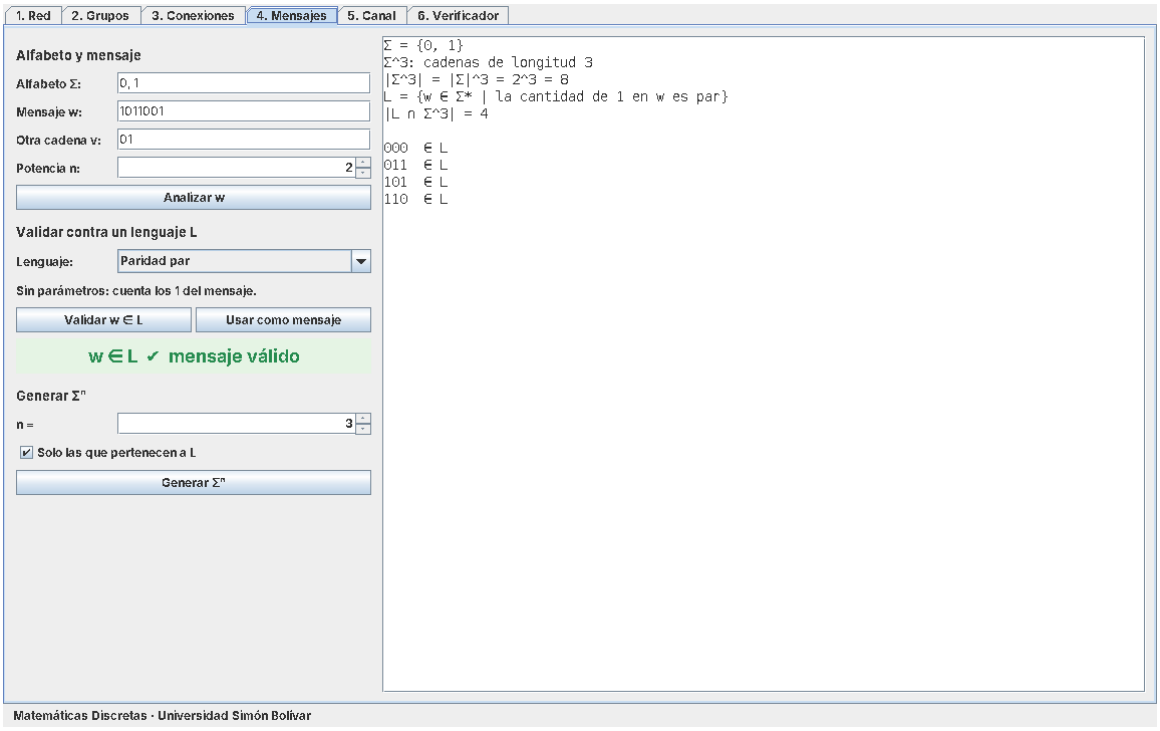


Figura 5. Módulo Mensajes: Σ³ filtrado por el lenguaje de paridad par.

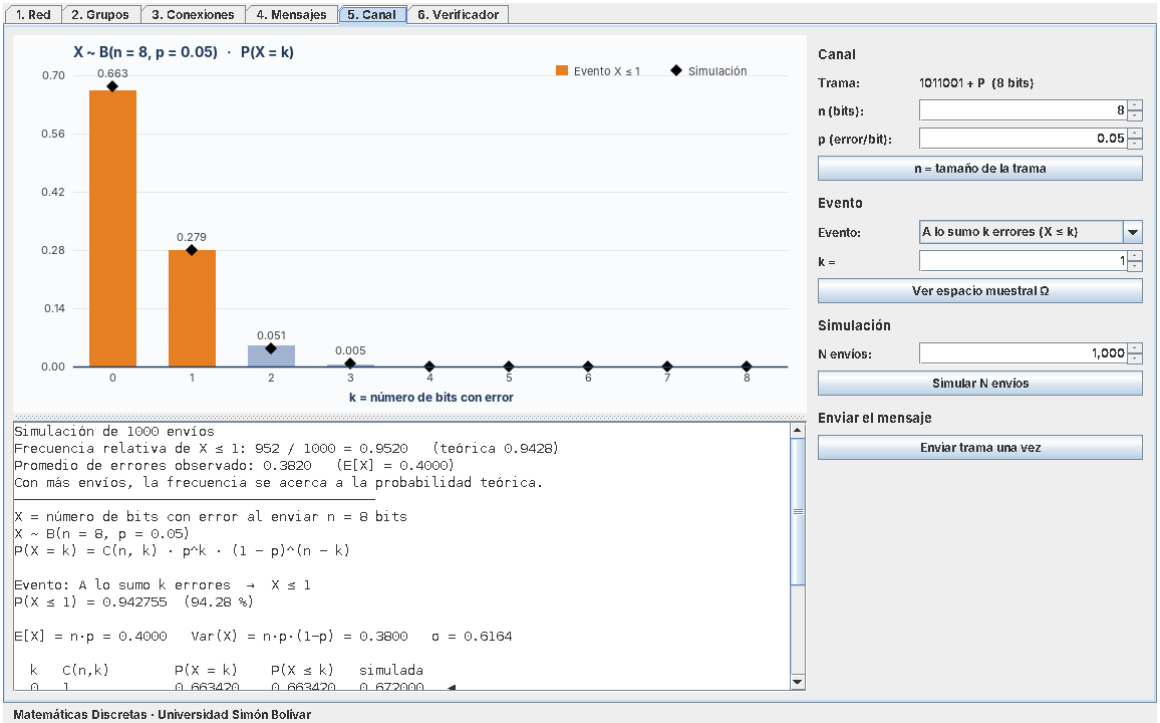


Figura 6. Módulo Canal: B(8; 0,05) con el evento X ≤ 1; los rombos son la simulación de 1.000 envíos.

En la simulación de la Figura 6, la frecuencia relativa de X ≤ 1 fue muy cercana a la teórica de 0,9428, y el promedio de errores se acercó a E[X] = 0,4. Con 100.000 envíos la diferencia fue menor al 0,2 %, lo que confirma la ley de los grandes números.

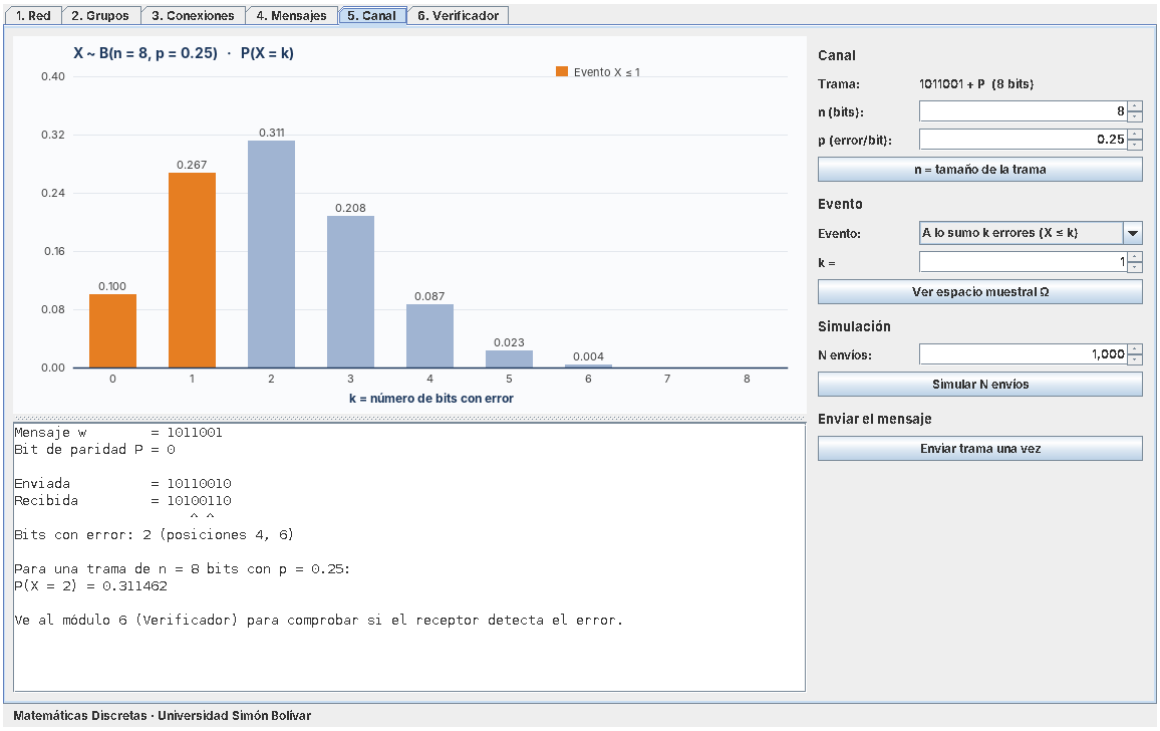


Figura 7. Envío de la trama 10110010 con p = 0,25: llegaron dañados dos bits.

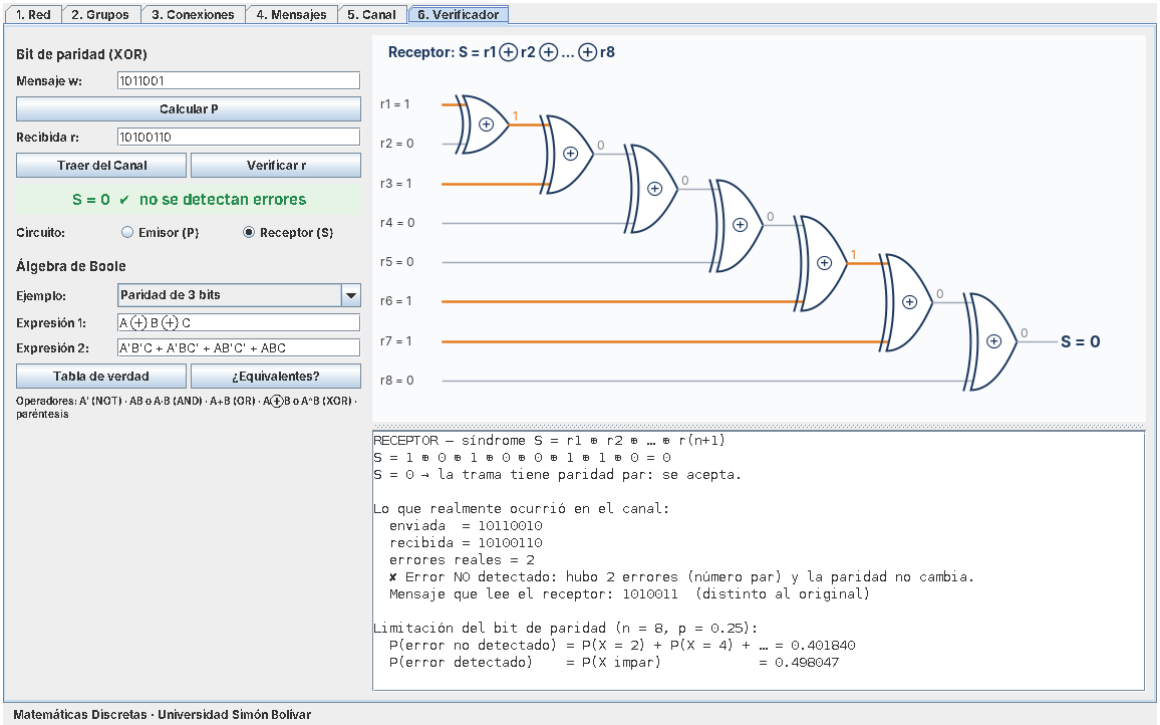


Figura 8. Módulo Verificador: con 2 errores el síndrome es S = 0 y el error no se detecta.

Las Figuras 7 y 8 muestran la principal limitación del bit de paridad: dos errores se cancelan y el receptor acepta un mensaje dañado. Con n = 8 y p = 0,25 esto ocurre con probabilidad 0,4018, valor que el propio programa calcula como P(X = 2) + P(X = 4) + P(X

= 6) + P(X = 8).

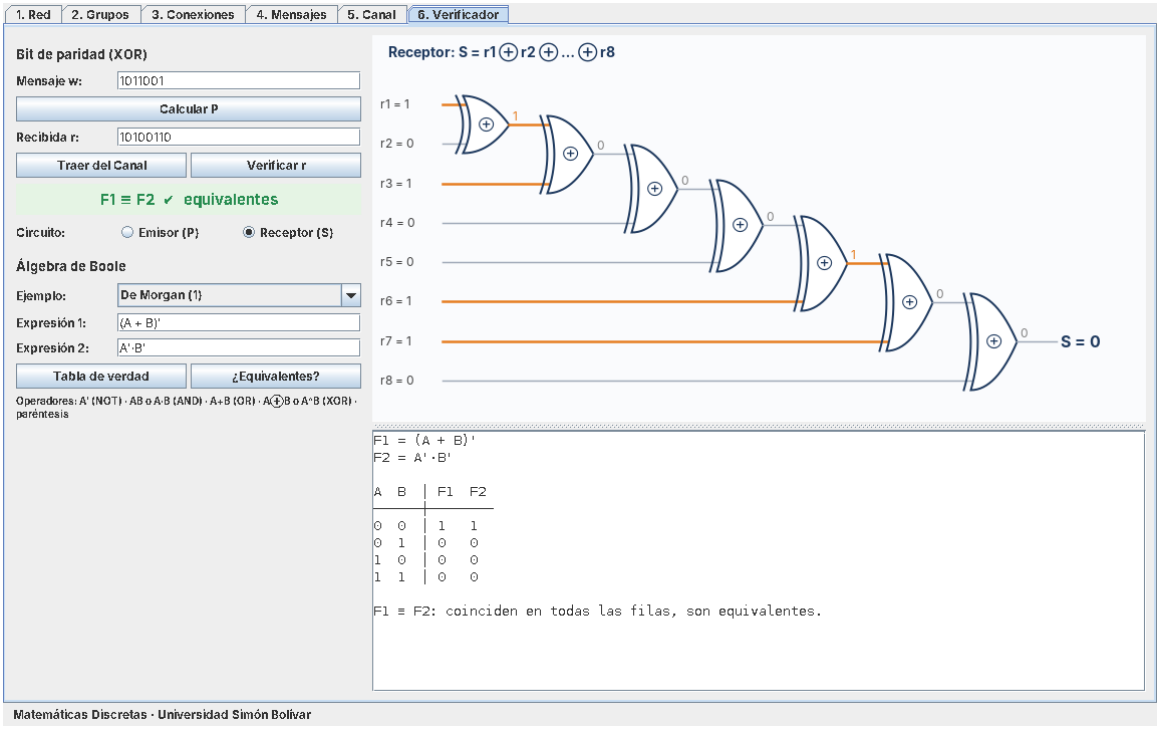


Figura 9. Comprobación de la ley de De Morgan con tablas de verdad.

8. Conclusiones

- Se cumplieron los seis objetivos específicos: cada tema del curso quedó implementado como un módulo funcional y conectado con los demás.
- Integrar los temas en un solo escenario mostró que no son independientes: el grafo define las relaciones del producto cartesiano, el lenguaje define qué mensajes son válidos, la binomial mide qué tan confiable es el canal y el álgebra de Boole permite detectar los errores.
- La simulación confirmó la teoría de probabilidad: con suficientes repeticiones las frecuencias observadas coinciden con la distribución binomial.
- El análisis del bit de paridad, combinado con la binomial, permitió cuantificar su limitación: detecta cualquier número impar de errores, pero no los pares.
- Separar la lógica de la interfaz facilitó probar cada algoritmo por separado y mantener el código ordenado.

9. Limitaciones y trabajo futuro

- La red es simulada; no hay comunicación real entre computadores.
- Para que la interfaz sea legible se limita el listado de Ω a $n \leq 5$, Σ^n a 4.096 cadenas, el circuito a 16 bits y las tablas de verdad a 6 variables.
- El canal no tiene en cuenta la ruta: un trabajo futuro sería que la probabilidad de error dependa del número de saltos calculado por Dijkstra, con probabilidad de éxito $(1 - p)^h$ en h enlaces.

- La paridad solo detecta errores. Se podría agregar un código de Hamming, que también los corrige.
- Se podría simplificar automáticamente las expresiones booleanas con mapas de Karnaugh o Quine–McCluskey.

10. Referencias

Rosen, K. H. (2004). *Matemática discreta y sus aplicaciones* (5.ª ed.). McGraw-Hill.

Grimaldi, R. P. (1998). *Matemáticas discreta y combinatoria* (3.ª ed.). Addison-Wesley Iberoamericana.

Johnsonbaugh, R. (2005). *Matemáticas discretas* (6.ª ed.). Pearson Educación.

Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2009). *Introduction to Algorithms* (3.ª ed.). MIT Press.

Tanenbaum, A. S. y Wetherall, D. J. (2012). *Redes de computadoras* (5.ª ed.). Pearson Educación.

Oracle. *The Java Tutorials: Creating a GUI With Swing*. https://docs.oracle.com/javase/tutorial/uiswing/

Repositorio del proyecto: https://github.com/NeyderJavier/NetDiscreta